Experiment 9

Aim: To implement a weather application using Flask that retrieves and displays weather information based on user input.

Lab Outcomes
By the end of this experiment, students will be able to:
- Create a Flask-based web application
- Accept user input (city name or zip code)
- Integrate a third-party weather API
- Display weather data in a user-friendly format

Objective
- To understand how Flask interacts with APIs by building a web application that fetches and displays real-time weather data.

Prerequisites
- Basic knowledge of Python
- Understanding of web applications
- Internet connection

Theory
Flask is a lightweight Python web framework used to build simple web applications. In this experiment, Flask is used to create a weather application that takes user input such as a city name and processes it on the server. The application sends a request to a weather API and receives data in JSON format. This data is then processed and displayed on a web page. The experiment demonstrates how web applications interact with external APIs and present real-time information to users. API integration allows applications to communicate with external services using HTTP requests.

Code : Weather Application using Flask

templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>Weather App</title>
  <style>
    body { font-family: Arial, sans-serif; background: #e0f2fe; display: flex; justify-content:
center; align-items: center; min-height: 100vh; margin: 0; }
    .card { background: white; padding: 2rem; border-radius: 12px; box-shadow: 0 4px 12px
rgba(0,0,0,0.1); width: 380px; }
    h2 { text-align: center; color: #0369a1; margin-bottom: 1.5rem; }
```

```
    .search { display: flex; gap: 0.5rem; margin-bottom: 1rem; }
    input { flex: 1; padding: 0.6rem 1rem; border: 1px solid #ccc; border-radius: 6px; font-size:
1rem; }
    button { padding: 0.6rem 1.2rem; background: #0284c7; color: white; border: none;
border-radius: 6px; font-size: 1rem; cursor: pointer; }
    button:hover { background: #0369a1; }
    .error { color: #dc2626; text-align: center; margin-top: 0.5rem; }
    .weather-box { text-align: center; margin-top: 1rem; }
    .city { font-size: 1.5rem; font-weight: bold; color: #1e293b; }
    .temp { font-size: 3rem; font-weight: bold; color: #0284c7; }
    .desc { font-size: 1rem; color: #64748b; margin-bottom: 1rem; }
    .details { display: flex; justify-content: space-around; background: #f0f9ff; border-radius:
8px; padding: 0.75rem; }
    .detail-item { text-align: center; }
    .detail-item span { display: block; font-size: 0.8rem; color: #94a3b8; }
    .detail-item strong { font-size: 1rem; color: #1e293b; }
  </style>
</head>
<body>
  <div class="card">
    <h2>⛅ Weather App</h2>
    <form method="POST">
      <div class="search">
        <input type="text" name="city" placeholder="Enter city name..." required />
        <button type="submit">Search</button>
      </div>
    </form>

    {% if error %}
      <p class="error">{{ error }}</p>
    {% endif %}

    {% if weather %}
      <div class="weather-box">
        <div class="city">{{ weather.city }}, {{ weather.country }}</div>
        <div class="temp">{{ weather.temp }}°C</div>
        <div class="desc">{{ weather.description }}</div>
        <div class="details">
          <div class="detail-item">
            <span>Feels Like</span>
            <strong>{{ weather.feels_like }}°C</strong>
          </div>
          <div class="detail-item">
            <span>Humidity</span>
            <strong>{{ weather.humidity }}%</strong>
          </div>
          <div class="detail-item">
            <span>Wind</span>
            <strong>{{ weather.wind }} km/h</strong>
          </div>
```

```
      </div>
    </div>
  {% endif %}
 </div>
</body>
</html>
```

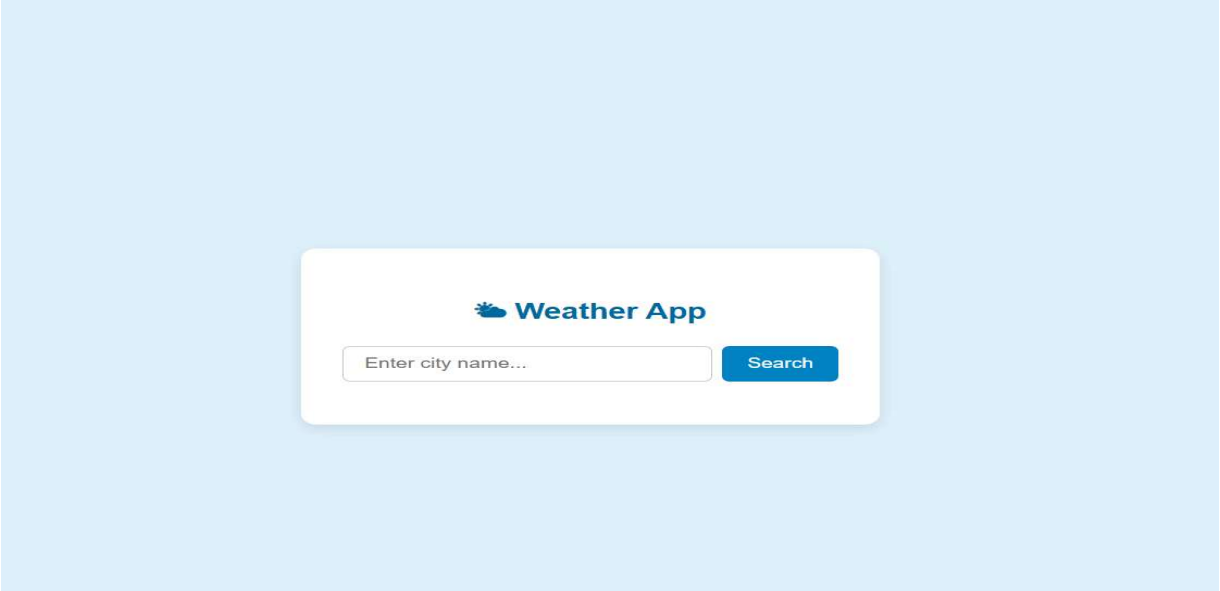
app.py

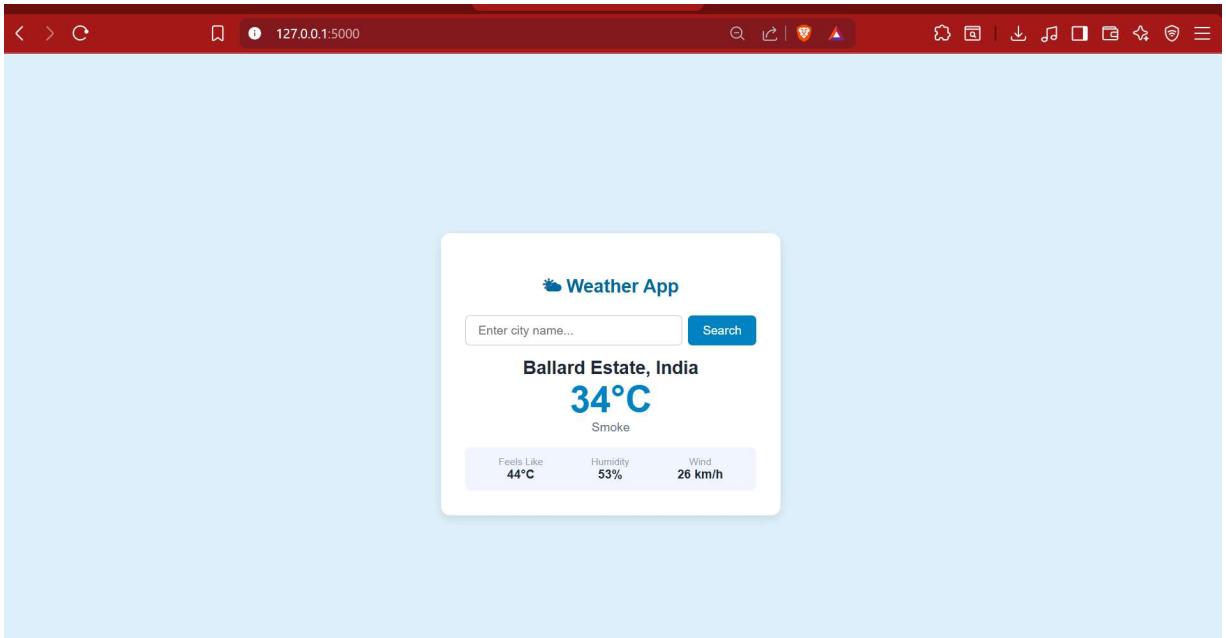
```
from flask import Flask, render_template, request
import requests

app = Flask(__name__)

@app.route("/", methods=["GET", "POST"])
def index():
    weather = None
    error = None
    if request.method == "POST":
        city = request.form.get("city", "").strip()
        if not city:
            error = "Please enter a city name."
        else:
            try:
                # wttr.in is free, no API key required
                url = f"https://wttr.in/{city}?format=j1"
                resp = requests.get(url, timeout=5)
                if resp.status_code == 200:
                    data = resp.json()
                    current = data["current_condition"][0]
                    area = data["nearest_area"][0]
                    weather = {
                        "city": area["areaName"][0]["value"],
                        "country": area["country"][0]["value"],
                        "temp": current["temp_C"],
                        "feels_like": current["FeelsLikeC"],
                        "humidity": current["humidity"],
                        "description": current["weatherDesc"][0]["value"],
                        "wind": current["windspeedKmph"],
                    }
                else:
                    error = "City not found. Please try another name."
            except requests.exceptions.RequestException:
                error = "Could not connect to weather service. Check your internet."
            except (KeyError, ValueError):
                error = "Could not parse weather data. Try a different city name."
    return render_template("index.html", weather=weather, error=error)

if __name__ == "__main__":
    app.run(debug=True)
```

requirements.txt

```
flask
requests
```

Output:-





Conclusion:-

This experiment helps in understanding how web applications are built using Flask and how external APIs can be integrated to fetch live data and display it to users.